

CLASS 04 · WEB DEVELOPMENT COURSE

HTML5 Forms & User Input

Part 1 — the fundamentals of collecting user data

How to build a form, decide where its data goes, choose the right input for each kind of answer, and label everything properly.

<form>

action & method

input types

<label> + for

placeholder vs value

Part 2 (next class): more input types, validation, fieldset/legend, select & textarea.

[Connecting to last class](#)

Quick Recap: <table> vs <form>

Both organize HTML into structured pieces — but they solve opposite problems.

table.html — a table

Name	Score
Ali	88
Sara	91

form.html — a form

Name

Submit

<TABLE> — DISPLAYS DATA

Shows structured data you already have, for the user to read. Read-only — nothing is sent back to the server.

<FORM> — COLLECTS DATA

Gathers new input from the user, interactively. Has action & method — the data IS sent to the server on submit.

In one line: a table shows information, a form asks for it.

Roadmap

Today's Roadmap

- 01 What is `<form>`? — the action & method attributes
- 02 Building a complete form skeleton
- 03 Text-based input types: text, password, email, number, date
- 04 checkbox & radio — letting users pick from options
- 05 `<label>` & the `for` attribute — connecting labels to fields
- 06 placeholder vs value — the classic mix-up

Structure

The <form> Element

<form> is the container that wraps every field you want to collect. It groups related inputs and knows how to submit them together.

- Everything inside it is submitted as one package
- A submit button (or pressing Enter) triggers that submission

```
<form action="/submit" method="post">  
  <!-- inputs go here -->  
  <button type="submit">Send</button>  
</form>
```

The action Attribute — Where Data Goes

action is a URL — the address the browser sends the form's data to when it's submitted.

- Can be a relative path (/login) or a full URL
- If you leave it out, the form submits to the current page's own URL

```
<form action="/login">
```

```
...
```

```
</form>
```

```
<!-- sends data to yoursite.com/login -->
```

form attributes

The method Attribute — GET vs POST

METHOD="GET"

Data is appended to the URL as a query string (?q=shoes). Visible, bookmarkable, has a length limit. Good for searches and filters.

METHOD="POST"

Data is sent inside the request body — not shown in the URL. No practical size limit. Used for logins, sign-ups, and anything sensitive.

Rule of thumb: use GET for retrieving/filtering data, POST for creating, changing, or submitting sensitive data.

[Live demo](#)

A Complete Form Skeleton

```
<form action="/signup" method="post">
  <label for="name">Name</label>
  <input id="name" type="text" name="name">

  <label for="email">Email</label>
  <input id="email" type="email" name="email">

  <button type="submit">Sign up</button>
</form>
```

preview

Name

Email

Sign up

Input types

Text-Based Input Types

The `type` attribute on `<input>` changes how the field behaves and what keyboard/validation the browser gives you.

`type="text"`

Free-form single line of text. This is also the default if you omit type entirely.

`type="password"`

Masks each character as a dot. Visually hidden only — not encrypted.

`type="email"`

Mobile shows an email-friendly keyboard; browser checks for a basic email shape on submit.

`type="number"`

Only accepts numbers; shows up/down spinner arrows. Supports min, max, step.

`type="date"`

Opens a native calendar picker. Value is always formatted YYYY-MM-DD.

Input types

type="checkbox" — Pick Any Number

An on/off toggle. You can check as many boxes as apply — perfect for "select all that apply" questions.

- checked — pre-selects the box when the page loads
- value — what gets submitted if the box is checked
- Unchecked boxes send nothing at all

preview

☒ HTML

☐ CSS

☐ JavaScript

```
<input type="checkbox" name="topic"
  value="html" checked> HTML
<input type="checkbox" name="topic"
  value="css"> CSS
```

Input types

type="radio" — Pick Exactly One

Lets the user choose only one option from a group — selecting one automatically deselects the others.

THE KEY RULE

All radio buttons in the same group must share the same name attribute — that's what links them together and makes them mutually exclusive.

```
<input type="radio" name="level"  
  value="beginner" checked> Beginner  
<input type="radio" name="level"  
  value="pro"> Pro
```

preview

- ☒ Beginner
- ☐ Intermediate
- ☐ Pro

Accessibility & usability

<label> & the for Attribute

<label> gives a field a real, clickable name.

`for="id"` connects a label to one specific input by matching its id — clicking the label text focuses (or toggles) that field.

BONUS

You can also wrap the <input> inside the <label> directly — then you don't need for or id at all.

```
<label for="age">Age</label>
<input id="age" type="number" name="age">

<!-- or, wrapped: -->
<label>Age <input type="number"></label>
```

The classic mix-up

placeholder vs value

PLACEHOLDER

Light gray hint text shown inside an empty field. Disappears the moment the user types. NOT submitted as data — it's just a visual example.

VALUE

The field's actual pre-filled content. Stays until the user changes it, and IS submitted as data when the form is sent.

```
<input type="text" placeholder="e.g. Jane Doe">  
<input type="text" value="Jane Doe">
```

DON'T DO THIS

Placeholder text is not a substitute for a `<label>` — it vanishes once typing starts and isn't reliably announced by every screen reader. Always pair it with a real label.

Wrap-up

Recap & What's Next

You now know:

- `<form>`, action, and method — where and how data is sent
- Text-based types: text, password, email, number, date
- checkbox (any number) vs radio (exactly one, shared name)
- `<label>` + for — connecting a label to its field
- placeholder (a hint, not submitted) vs value (real, submitted data)

COMING UP IN PART 2

More input types (tel, url, range, file, color), validation attributes like required and pattern, `<fieldset>/<legend>` for grouping, and `<select>/<textarea>`.